Université Nationale des Sciences, Technologies, Ingénierie et Mathématiques (UNSTIM) d'Abomey

École Nationale Supérieure de Génie Mathématique et Modélisation (ENSGMM)

Unité d'enseignement : Système d'Information

Modèle NoSQL orienté colonnes

Column-Family / Wide-Column

Groupe 4

AGRO Rachidi
ANATO Krisland
HOUNSOUNOU Marius

Sous la supervision de

Dr (MC). KONNON Abel

Année universitaire 2026-2027

Plan

1. Contexte et problématique
2. Le modèle de données
3. Architecture distribuée
4. Performances, systèmes et usages
5. Étude de cas : capteurs IoT avec Cassandra
6. Mesures sur dix ans : Cassandra face à PostgreSQL
7. Étude comparative
8. Du relationnel au Wide-Column
9. Conclusion

Partie 1

Contexte et problématique

D'où viennent les bases orientées colonnes ?

- Le relationnel (Codd, 1970) repose sur des jointures et des transactions, sur un serveur unique.
- Un serveur ne grandit pas indéfiniment : avec des millions d'écritures continues, on répartit les données sur de nombreuses machines ordinaires.
- Il faut alors continuer à fonctionner quand l'une d'elles tombe en panne.

2006 Google : Bigtable
lignes triées, familles de colonnes

2007 Amazon : Dynamo
clé-valeur sans maître, disponible

2008 Facebook : Cassandra
modèle Bigtable + architecture Dynamo

2010 Projet Apache
aujourd'hui Cassandra 5

Problématique

Comment les bases orientées colonnes gèrent-elles efficacement
des volumes massifs de données distribuées,
et quels compromis impliquent-elles ?

- Exemple suivi tout au long de l'exposé : un réseau de capteurs de température et d'humidité.
- Avec ces bases, on part des requêtes que l'application devra exécuter ; en relationnel, on part des données.

Partie 2

Le modèle de données

Deux sens du mot « colonne »

1. Relationnel (stockage par ligne) : *une ligne = un bloc*

1 | Ali | Cotonou | 1500	2 | Bio | Abomey | 2300	3 | Zola | Ouidah | 850

2. Stockage colonnaire (analytique) : *une colonne = un bloc*

1 | 2 | 3	Ali | Bio | Zola	Cotonou | Abomey | Ouidah	1500 | 2300 | 850

3. Wide-Column / Column-Family (Cassandra, HBase) : *une clé + des cellules triées*

clé S001	ts1 : temp=27.4	ts1 : hum=61	ts2 : temp=27.6	ts2 : hum=60

- Le stockage colonnaire (ClickHouse, Redshift, Parquet) sert l'analyse : peu de colonnes lues sur beaucoup de lignes.
- Le modèle Wide-Column (Bigtable, Cassandra, HBase, ScyllaDB) sert la distribution et l'écriture massive. C'est notre sujet.

Le modèle de Bigtable

$$(\textit{clé de ligne},\ \textit{famille :colonne},\ \textit{horodatage}) \longmapsto \textit{valeur}$$

- Les lignes sont triées par clé : les lectures par plage sont efficaces.
- Les familles de colonnes sont déclarées à l'avance ; les colonnes s'ajoutent librement à l'intérieur.
- Une cellule peut garder plusieurs versions, distinguées par leur horodatage.
- La table est éparse : une colonne sans valeur ne coûte rien, deux lignes peuvent avoir des colonnes différentes.

Le vocabulaire de Cassandra

Élément	Rôle
Cluster	Ensemble des nœuds
Keyspace	Regroupe les tables, fixe la réplication
Table	Partitions décrites par un schéma
Partition	Lignes de même clé de partition ; unité de répartition
Ligne	Repérée par sa clé de clustering
Cellule	Valeur + horodatage (+ TTL)

Clé primaire en deux parties

`PRIMARY KEY ((sensor_id, day), ts)`

La clé de partition `(sensor_id, day)` choisit les nœuds qui stockent la donnée.

La clé de clustering `ts` trie les lignes dans la partition.

Pourquoi « wide » ?

Une partition peut contenir un très grand nombre de lignes, triées par la clé de clustering.

Clé de partition (S001, 2026-10-05)			
ts=08:00:20	temp=27.9	hum=60.5	bat=88
ts=08:00:10	temp=27.6	hum=60.8	bat=88
ts=08:00:00	temp=27.4	hum=61.0	bat=88

… des milliers de lignes, de la plus récente à la plus ancienne (`ts DESC`)

- Toutes les mesures d'un capteur pour un jour sont stockées ensemble.
- Dans Cassandra, c'est la partition qui joue le rôle de ligne extensible.

Relationnel, colonnaire, Wide-Column

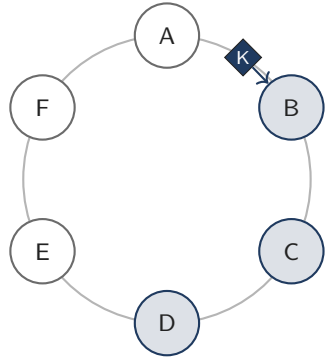
Critère	Relationnel	Stockage colonnaire	Wide-Column
Unité de stockage	La ligne	La colonne	La partition
Usage visé	Transactions (OLTP)	Analyse (OLAP)	Écriture et lecture massives, distribuées
Schéma	Fixe	Fixe	Familles fixes, colonnes souples
Jointures	Natives	Oui	Non, évitées par la modélisation
Transactions	ACID complètes	Limitées	Par partition, ou *lightweight*
Distribution	Difficile	Selon le système	Native
Modélisation	À partir des données	À partir des analyses	À partir des requêtes
Exemples	PostgreSQL, MySQL	ClickHouse, Redshift	Cassandra, HBase, ScyllaDB

Partie 3

Architecture distribuée

Partitionnement et réplication

- Pas de serveur maître : tous les nœuds ont le même rôle.
- Murmur3 transforme la clé de partition en un *token*, entre -2^{63} et $2^{63} - 1$.
- Chaque nœud possède des plages de l'anneau (*vnodes*) ; ajouter un nœud ne déplace qu'une fraction des données.
- Le facteur de réplication (RF) fixe le nombre de copies. Aucune copie n'est principale.



Clé K dans la plage de B ; avec RF = 3, copies sur B, C et D

Cohérence ajustable

Niveau	Copies qui doivent répondre
`ONE`	Une seule : le plus rapide
`QUORUM`	La majorité : 2 sur 3 si RF $= 3$
`LOCAL_QUORUM`	La majorité dans le datacenter local
`ALL`	Toutes : une panne bloque la requête

Lire la dernière écriture

$$R + W > RF$$

Avec RF $= 3$, écrire et lire en `QUORUM` : $2 + 2 = 4 > 3$.

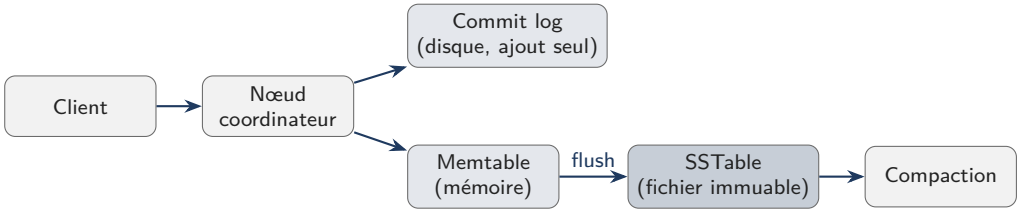
- Théorème CAP : lors d'une coupure réseau, on renonce à la cohérence ou à la disponibilité. Cassandra privilégie la disponibilité par défaut.
- PACELC : même sans panne, il reste un compromis entre latence et cohérence.

Tolérance aux pannes

Cassandra n'a pas de point de défaillance unique. Quatre mécanismes se relaient :

- le *gossip* : chaque nœud échange avec quelques voisins l'état de tous les autres et détecte les pannes ;
- le *hinted handoff* : un nœud garde les écritures destinées à un nœud absent et les lui transmet à son retour (3 heures par défaut) ;
- le *read repair* : si deux copies diffèrent à la lecture, la plus récente est propagée ;
- l'*anti-entropy repair* : `nodetool repair` compare les copies par arbres de Merkle.

Chemin d'écriture et de lecture



- Le client est confirmé dès que le commit log et la memtable sont à jour : aucune lecture préalable, aucune modification sur place (LSM-tree).
- À la lecture, la memtable et les SSTables sont fusionnées ; la cellule la plus récente l'emporte.
- Un filtre de Bloom signale qu'une SSTable ne contient pas la partition et évite de la lire.
- Une suppression écrit une *tombstone*, effacée à la compaction après `gc_grace_seconds` (10 jours).

Partie 4

Performances, systèmes et usages

Ce que l'on gagne, ce que l'on paie

Avantage	Contrepartie
Montée en charge par ajout de machines, haute disponibilité	Cohérence forte non automatique : il faut choisir les niveaux
Écritures très rapides (LSM-tree)	Lectures plus coûteuses, compaction en arrière-plan
Aucune jointure à exécuter à la lecture	Duplication : une table par type de requête
Schéma souple	Une requête non prévue est coûteuse ou refusée

- La complexité passe de la lecture (jointures, agrégations) à la conception des tables et à l'écriture.
- Une mauvaise conception se corrige rarement par une requête plus astucieuse : il faut souvent recréer la table.

Principaux systèmes

	Cassandra	HBase	ScyllaDB
Langage	Java	Java	C++
Architecture	Pair-à-pair	Maître + régions, HDFS	Pair-à-pair, un fragment par cœur
Interface	CQL	API HBase, shell	CQL (compatible Cassandra)
Cohérence	Ajustable	Forte par ligne	Ajustable
Point fort	Disponibilité, multi-datacenter	Intégration Hadoop et Spark	Performance par machine
Limite	Réglage de la JVM, compaction	Déploiement complexe	Écosystème plus restreint

Google Cloud Bigtable, service géré issu du système d'origine, s'utilise avec l'API de HBase.

Cas d'utilisation

Domaine	Partition type
IoT	(capteur, jour)
Journaux (logs)	(service, heure)
Séries temporelles	(métrique, jour)
Télécommunications	(abonné, mois)
Big Data	Clé bien répartie

Moins adapté

Transactions ACID entre tables (comptabilité, virements).

Analyse libre, quand les questions ne sont pas connues d'avance.

Petits volumes : un serveur relationnel suffit.

Les bons usages réunissent beaucoup d'écritures, des requêtes connues d'avance et un besoin de distribution.

Partie 5

Étude de cas : capteurs IoT avec Cassandra

Partir des requêtes, pas des données

Des capteurs envoient une mesure (température, humidité, batterie) toutes les 10 secondes.

N°	Requête métier
Q1	Dernières mesures d'un capteur pour un jour
Q2	Mesures entre deux heures
Q3	Dernière mesure de chaque capteur
Q4	Alertes ($> 35\,°C$) d'une zone pour un jour
Q5	Indicateurs d'une journée
Q6	Évolution des indicateurs sur une période

Table	Sert
`readings_by_sensor_day`	Q1, Q2, Q5
`last_reading_by_sensor`	Q3
`alerts_by_zone_day`	Q4
`daily_stats_by_sensor`	Q6

Six requêtes, quatre tables : une même mesure est écrite plusieurs fois.

La table principale

```
CREATE TABLE readings_by_sensor_day (
  sensor_id text,
  day date,
  ts timestamp,
  temperature double,
  humidity double,
  battery_pct int,
  PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC)
  AND default_time_to_live = 7776000
  AND compaction = {
    'class': 'TimeWindowCompactionStrategy',
    'compaction_window_unit': 'DAYS',
    'compaction_window_size': 1 };
```

- Le jour dans la clé de partition borne sa taille : $6 \times 60 \times 24 = 8\,640$ lignes par jour au lieu de 3,15 millions par an (*bucketing*).
- `ts DESC` : Q1 lit les premières lignes, sans tri.
- Expiration après 90 jours et compaction par fenêtres de temps.
- Keyspace répliqué sur 3 nœuds (RF = 3).

Requêtes avec la clé de partition

```
cqlsh:iot> SELECT ts, temperature, humidity FROM readings_by_sensor_day
  WHERE sensor_id = 'S001' AND day = '2026-10-05' LIMIT 10;

 ts                              | temperature | humidity
---------------------------------+-------------+----------
 2026-10-05 08:00:20.000000+0000 |        27.9 |     60.5
 2026-10-05 08:00:10.000000+0000 |        27.6 |     60.8
 2026-10-05 08:00:00.000000+0000 |        27.4 |       61

(3 rows)
cqlsh:iot> SELECT ts, temperature FROM readings_by_sensor_day
  WHERE sensor_id = 'S001' AND day = '2026-10-05'
    AND ts >= '2026-10-05 08:00:00+0000' AND ts < '2026-10-05 12:00:00+0000';

 ts                              | temperature
---------------------------------+-------------
 2026-10-05 08:00:20.000000+0000 |        27.9
 2026-10-05 08:00:10.000000+0000 |        27.6
 2026-10-05 08:00:00.000000+0000 |        27.4

(3 rows)
cqlsh:iot>
```

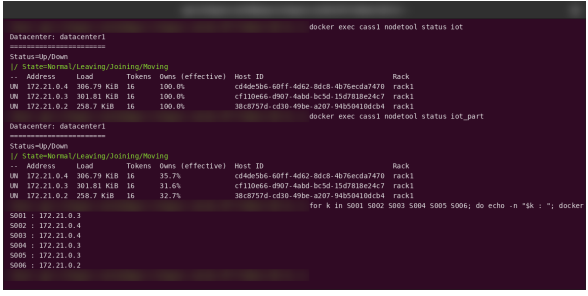
- Q1 renvoie les mesures dans l'ordre décroissant, sans tri.
- Q2 lit une plage contiguë de la partition.
- Cassandra sait quels nœuds interroger, car la requête donne la clé de partition complète.

Une requête non prévue est refusée

```
cqlsh:iot> SELECT * FROM readings_by_sensor_day WHERE temperature > 35;
InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query
as it might involve data filtering and thus may have unpredictable performance. If you want to
execute this query despite the performance unpredictability, use ALLOW FILTERING"
cqlsh:iot>
```

- Sans clé de partition, il faudrait parcourir toutes les partitions de tous les nœuds.
- `ALLOW FILTERING` force l'exécution : négligeable sur quelques lignes, très lourd sur des millions.
- La bonne réponse est une table conçue pour la question, ici `alerts_by_zone_day`.

Réplication, partitionnement et panne



- RF = 3 sur 3 nœuds : chaque nœud possède 100 % des données de `iot`.
- RF = 1 : chaque capteur est confié au nœud que désigne le hachage de son identifiant.
- Après `docker stop cass3`, Q1 réussit encore en `QUORUM` : deux copies sur trois suffisent.

Partie 6

Mesures sur dix ans : Cassandra face à PostgreSQL

Protocole

- 50 capteurs, 5 zones, une mesure toutes les 15 minutes.
- 1 752 000 mesures par an ; **17 529 600** sur dix ans (2016 à 2026).
- Mêmes données dans les deux bases, générées avec une graine fixe.
- Mesures après 1, 2, 3, 5 et 10 ans d'historique.

- Cassandra 5.0.9, un nœud, même modèle que l'étude de cas.
- PostgreSQL 15.19, schéma normalisé, clé primaire `(sensor_id, ts)`.
- Médiane de 50 exécutions ; lectures à froid après redémarrage et vidage des caches.
- Scripts et résultats bruts fournis avec le rapport.

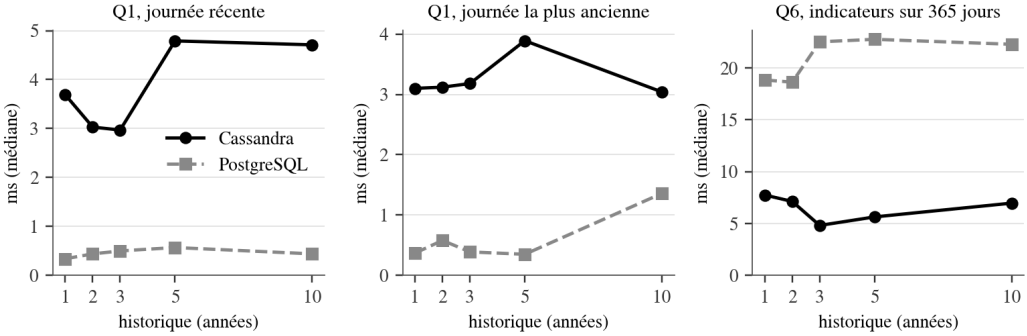
Volume stocké

```
docker exec cass-bench nodetool tablestats iot_bench.readings_by_sensor_day | grep -E "Table:|SSTable count:|Space used \(live\)|Number of partitions|Compacted partition mean"
                Table: readings_by_sensor_day
                SSTable count: 5
                Old SSTable count: 0
                Space used (live): 348676950
                Number of partitions (estimate): 180818
                Compacted partition mean bytes: 3984
docker exec pg-bench psql -U postgres -d iot -c "SELECT count(*) AS mesures, min(ts)::date AS debut, max(ts)::date AS fin, pg_size_pretty(pg_total_relation_size('readings')) AS taille FROM readings;"
 mesures  |   debut    |    fin     | taille
----------+------------+------------+--------
 17529600 | 2016-10-15 | 2026-10-14 | 1678 MB
(1 row)
```

Historique	1 an	2 ans	3 ans	5 ans	10 ans
Cassandra (Mo)	35	69	104	174	349
PostgreSQL, table + index (Mo)	175	350	526	876	1 755

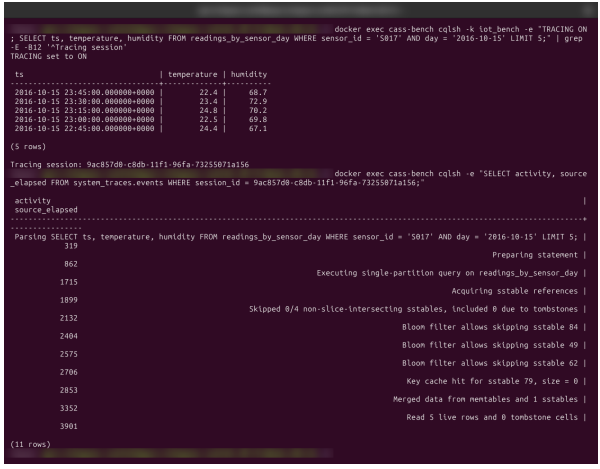
Cinq fois moins de place pour Cassandra : compression LZ4, clé de partition écrite une seule fois.

Temps de lecture selon le volume



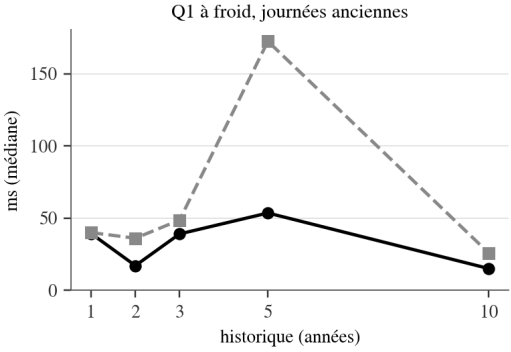
- Cassandra reste entre 3 et 5 ms de 1 à 10 ans : la partition contient toujours 96 lignes.
- PostgreSQL lit plus vite sur une seule machine, mais Q1 ancien passe de 0,37 à 1,36 ms.
- Q6, prévue d'avance : 7 ms (table précalculée) contre 22 ms (agrégation à chaque appel).

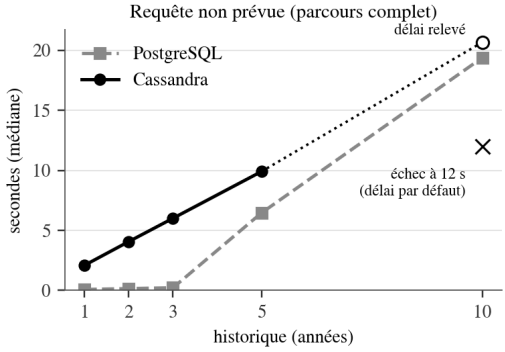
Retrouver une journée de 2016



```
                                                
```

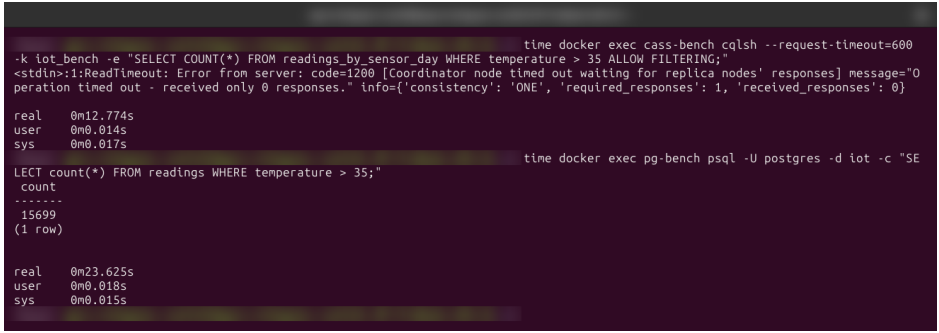
Lecture à froid et requête non prévue





- À froid, Cassandra est à chaque volume aussi rapide ou plus rapide (15 à 55 ms contre 26 à 172 ms).
- La requête non prévue croît d'environ 2 s par an chez Cassandra et échoue à dix ans.

La requête non prévue à dix ans



- Cassandra interrompt le parcours au bout de 12 s (`ReadTimeout`), six fois sur six.
- Avec le délai relevé, elle trouve les mêmes 15 699 mesures en 20,7 s.
- PostgreSQL répond toujours, mais en 16 à 24 s : la table ne tient plus en mémoire.

Synthèse des mesures

Critère	Cassandra	PostgreSQL
Lecture par clé, données en mémoire	3 à 5 ms, constant de 1 à 10 ans	0,3 à 1,6 ms, en hausse avec le volume
Donnée ancienne	Même coût qu'une donnée récente	Même coût qu'une donnée récente
Lecture à froid	15 à 55 ms	26 à 172 ms
Question prévue (Q6)	7 ms	22 ms
Question imprévue (10 ans)	Échec à 12 s ; 20,7 s délai relevé	16 à 24 s
Écritures une par une	**2 504** par seconde	305 par seconde
Espace disque (10 ans)	**349 Mo**	1 755 Mo

Mesures sur une seule machine : l'atout principal de Cassandra, l'ajout de nœuds, n'y apparaît pas.

Partie 7

Étude comparative

Relationnel et Wide-Column, critère par critère

Critère	Relationnel (PostgreSQL)	Wide-Column (Cassandra)
Conception	À partir des données	À partir des requêtes
Schéma de l'étude de cas	2 tables, sans duplication	4 tables, mesures dupliquées
Jointures	À la lecture	Remplacées par la duplication
Transactions	ACID sur plusieurs tables	Lot atomique, transactions légères
Montée en charge	Serveur plus puissant	Ajout de nœuds
Lecture par clé (mesurée)	0,3 à 1,6 ms, en hausse	3 à 5 ms, constante sur 10 ans
Écritures unitaires (mesurées)	305 /s	2 504 /s
Espace disque, 10 ans (mesuré)	1 755 Mo	349 Mo
Question imprévue (mesurée)	Exécutée, 16 à 24 s	Échec à 12 s
Nouveau besoin	Nouvelle requête	Nouvelle table + migration

Études de cas comparatives

Cas	Conception relationnelle	Conception Wide-Column	Retenu
Capteurs IoT	`readings(sensor_id, ts)` indexée	`((sensor_id, day), ts)`	Wide-Column
Messagerie	`messages` indexée par conversation	`((conversation_id, mois), date)`	Wide-Column
Commandes en ligne	Stock, paiement, lignes dans une transaction ACID	Plusieurs tables sans transaction commune	Relationnel
Journaux applicatifs	Table qui grossit, purge par `DELETE`	`((service, heure), date)` + TTL	Wide-Column

- Le choix dépend de la nature des accès plus que de la taille.
- Requêtes connues, écritures nombreuses, distribution : Wide-Column.
- Transactions entre entités, questions changeantes : relationnel. Beaucoup de systèmes combinent les deux.

Partie 8

Du relationnel au Wide-Column

Peut-on passer du relationnel au Wide-Column ?

Oui. Il n'y a pas de conversion automatique : on refait le schéma en partant des requêtes, puis on recopie les données.

1. Inventorier les requêtes de l'application (code, `pg_stat_statements`).
2. Garder le modèle conceptuel (entités, associations).
3. Concevoir une table par requête fréquente.
4. Choisir les clés : égalités en partition, plages et tri en clustering, période pour borner.
5. Fixer TTL, compaction, réplication.
6. Migrer l'historique : jointures et agrégations faites à l'export.
7. Vérifier : comptages et mêmes réponses dans les deux bases.
8. Basculer : double écriture ou capture des changements (Debezium, Kafka), puis lectures sur Cassandra.

Règles de correspondance

Relationnel	Wide-Column (Cassandra)
Table normalisée	Plusieurs tables dénormalisées, une par requête
Jointure, clé étrangère	Colonnes recopiées dans la table qui en a besoin
Association 1-N	Lignes filles dans la partition du parent, triées par clustering
Association N-N	Deux tables, une par sens de lecture
`GROUP BY` fréquent	Table précalculée
Index secondaire	Table dédiée à la requête
Contrainte d'intégrité	Contrôle par l'application
Transaction sur plusieurs tables	`BATCH` logué ou transaction légère (`IF NOT EXISTS`)
Identifiant auto-incrémenté	`uuid` ou `timeuuid`

Application à notre schéma

Table Cassandra	Export SQL
`readings_by_sensor_day`	`ts::date`
`alerts_by_zone_day`	jointure `sensors`
`daily_stats_by_sensor`	`GROUP BY`
`last_reading_by_sensor`	`LATERAL`

Le travail que PostgreSQL faisait à chaque lecture est fait une seule fois, à l'export.

```
-- psql : export de la table des alertes
\copy (SELECT s.zone, r.ts::date, ...,
         r.sensor_id, r.temperature
       FROM readings r JOIN sensors s
         ON s.sensor_id = r.sensor_id
       WHERE r.temperature > 35)
  TO '/tmp/alerts.csv' WITH (FORMAT csv)
```

```
// cqlsh : import
COPY iot_migre.alerts_by_zone_day
  (zone, day, ts, sensor_id, temperature)
  FROM '/tmp/alerts.csv';
```

Procédure complète : dossier `migration/` (`migrer.sh`).

Partie 9

Conclusion

Conclusion

- Les bases Wide-Column stockent de très grands volumes arrivant en continu, sur des machines ordinaires, sans point de défaillance unique.
- Elles reposent sur le partitionnement par hachage, la réplication sans maître, la cohérence réglable et l'écriture en ajout seul.
- En retour, il faut connaître les requêtes avant de créer les tables, accepter la duplication et renoncer aux jointures.
- Sur dix ans de mesures, le temps de lecture de Cassandra n'a pas bougé avec le volume ; la requête imprévue a fini par échouer.
- En pratique, on associe souvent une base Wide-Column pour l'ingestion et un entrepôt colonnaire pour l'analyse.

Merci pour votre attention

Questions et échanges

Références principales

- F. Chang, J. Dean, S. Ghemawat et al. *Bigtable : A Distributed Storage System for Structured Data*. OSDI, 2006.
- G. DeCandia, D. Hastorun et al. *Dynamo : Amazon's Highly Available Key-value Store*. SOSP, 2007.
- A. Lakshman, P. Malik. *Cassandra : A Decentralized Structured Storage System*. ACM SIGOPS OSR, 44(2), 2010.
- P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil. *The Log-Structured Merge-Tree (LSM-Tree)*. Acta Informatica, 1996.
- S. Gilbert, N. Lynch. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. 2002.
- D. Abadi. *Consistency Tradeoffs in Modern Distributed Database System Design*. IEEE Computer, 2012.
- A. Chebotko, A. Kashlev, S. Lu. *A Big Data Modeling Methodology for Apache Cassandra*. IEEE BigData Congress, 2015.
- Documentation Apache Cassandra : `cassandra.apache.org/doc/latest`